

Informe #2

Definición de la Metodología del Proyecto



Integrantes:
Owuen Yagual Panimboza
Stiven Rivera Palma

12 de Junio, 2026

Chapter 2

Metodología

2.1 Análisis de los Datos

El sistema opera principalmente sobre datos de video RGB en tiempo real capturados por una cámara convencional. A continuación se presenta el análisis de los datasets de referencia utilizados para la evaluación del desempeño, así como las características de los datos que maneja cada módulo del proyecto.

2.1.1 Datasets de Referencia

Para la evaluación y benchmarking del sistema se utilizarán datasets públicos consolidados en el campo de estimación de pose humana. La elección de estos conjuntos de datos se justifica por su amplia adopción en la literatura científica y su compatibilidad con el modelo MediaPipe Pose. [?, ?]

Dataset	Tipo	Imágenes Videos	/	Landmarks	Uso en el proyecto
COCO Keypoints	Imágenes estáticas RGB	328K imágenes, 2.5M instancias		17 keypoints	Benchmark de referencia para evaluar precisión de MediaPipe Pose
LSP	Imágenes RGB de deporte	2000 imágenes		14 keypoints	Validación del cálculo angular en poses con alta variabilidad

Table 1. – Datasets de referencia para evaluación del sistema.

2.1.2 Análisis de Datos por Módulo

Cada módulo del sistema procesa un tipo de dato diferente. La siguiente tabla detalla las características de los datos de entrada, las propiedades relevantes y el preprocesamiento necesario en cada etapa del pipeline.

Módulo	Tipo de dato de entrada	Características clave	Preprocesamiento necesario
M1. Captura	Frame BGR de video (uint8, 3 canales)	Resolución variable; posible ruido de sensor; iluminación no controlada	Redimensionado a 640×480 ; conversión BGR→RGB; filtro gaussiano opcional
M2. MediaPipe Pose	Frame RGB normalizado	Sujeto visible; cuerpo parcialmente ocluido hasta aproximadamente un 30%	Reflejo horizontal opcional (modo selfie); ajuste de <code>min_detection_confidence</code> ≥ 0.7 ; extracción de landmarks 11, 13, 15, 17, 19, 21 (hombro, codo, muñeca, nudillo índice, punta índice y punta pulgar derechos)
M3. Ángulos	Landmarks 2D (float32, $x, y \in [0, 1]$)	Coordenadas normalizadas de 7 landmarks; fluctuaciones de ± 2 a ± 5 px por ruido de BlazePose; distancia euclidiana entre landmarks 19 y 21 para apertura del gripper	Desnormalización a píxeles; filtro media móvil $w = 5$; verificación de visibilidad > 0.5
M4. Brazo virtual	Tripleta de ángulos (float64, grados)	Valores continuos $[0, 180]$; posibles discontinuidades en límites angulares	Clipping a rango físico válido por articulación; interpolación lineal para suavizado extra
M5. Visualización	Frame BGR + overlays compuestos	Alta variabilidad según iluminación y fondo	Sin preprocesamiento adicional; medición de FPS por diferencia de timestamps

Table 2. – Análisis de datos de entrada, características y preprocesamiento por módulo.

2.1.3 Análisis de Confiabilidad y Compatibilidad

Se analiza la confiabilidad de las principales soluciones existentes y su compatibilidad con los requisitos del proyecto. MediaPipe Pose (BlazePose) ha sido validado con una diferencia de precisión inferior al 10% respecto a sistemas IMU de referencia en estimación de movimientos corporales. [?]

En términos de compatibilidad tecnológica, el stack seleccionado (Python + OpenCV + MediaPipe) presenta las siguientes garantías: (a) compatibilidad multiplataforma (Windows, Linux, macOS) sin modificación del código; (b) ausencia de dependencias de GPU, lo que elimina problemas de compatibilidad de drivers; (c) todas las librerías son de código abierto con licencias Apache 2.0 o BSD, sin restricciones de uso académico; y (d) versiones estables con soporte activo a la fecha del proyecto (2026). [?, ?, ?]

- **MediaPipe Pose v0.10+:** estable desde 2022, soportado en Python 3.8–3.12, compatible con OpenCV 4.x. Latencia de inferencia < 30 ms en CPU. [?]
- **OpenCV v4.8+:** librería de visión computacional más usada globalmente, con más de 20 años de desarrollo activo y amplia documentación. [?]
- **NumPy v1.24+:** librería estándar de álgebra numérica en Python, prerequisite de todas las demás librerías del stack.

2.2 Alcance del Proyecto

El alcance del proyecto queda delimitado por los siguientes criterios: el sistema procesa video de una sola persona visible en cámara (single-person pose estimation), en entornos con iluminación entre 50 y 500 lux, con el sujeto a una distancia de 1–3 metros de la cámara. El brazo robótico virtual es una simulación 2D renderizada en pantalla y no controla hardware físico. [?]

2.2.1 Requerimientos

Requerimientos Funcionales

ID	Requerimiento	Prioridad	Módulo(s)
RF-01	El sistema debe capturar video en tiempo real desde una cámara RGB a mínimo 30 FPS	Alta	M1
RF-02	El sistema debe detectar la pose del cuerpo humano usando MediaPipe Pose con una confianza mínima de 0.7, extrayendo únicamente los landmarks 11, 13, 15, 17, 19 y 21 correspondientes al hombro, codo, muñeca, nudillo, punta del índice y punta del pulgar derechos	Alta	M2, M3
RF-03	El sistema debe calcular los ángulos de hombro, codo y muñeca con un error máximo de $\pm 5^\circ$	Alta	M3
RF-04	El sistema debe aplicar filtro de media móvil de ventana $w = 5$ para suavizar los ángulos calculados	Media	M3
RF-05	El sistema debe renderizar un brazo robótico virtual de 3 GDL sincronizado con el movimiento en tiempo real	Alta	M4
RF-06	El sistema debe mostrar los ángulos articulares y las métricas de FPS superpuestas en el frame de video	Media	M5
RF-07	El sistema debe exportar el video procesado a un archivo MP4 cuando el usuario lo requiera	Baja	M5
RF-08	El sistema debe procesar al menos 20 FPS en hardware de CPU estándar (sin GPU)	Alta	M1, M5
RF-09	El sistema debe calcular la apertura del gripper como la distancia euclidiana normalizada entre la punta del índice (landmark 19) y la punta del pulgar (landmark 21), mapeada al rango [0%, 100%]	Alta	M3, M4

Table 3. – Requerimientos funcionales del sistema.

Requerimientos No Funcionales

ID	Requerimiento	Criterio de aceptación
RNF-01	Rendimiento: ≥ 20 FPS en CPU Intel Core i5 con 8 GB RAM	FPS promedio medido sobre 500 frames consecutivos ≥ 20
RNF-02	Precisión angular: error medio (MAE) $\leq 5^\circ$ respecto a medición de referencia	MAE calculado sobre 100 poses con transportador digital de referencia
RNF-03	Robustez: detección estable con variaciones de iluminación entre 50 y 500 lux	Tasa de detección exitosa $\geq 90\%$ en el rango de iluminación especificado
RNF-04	Portabilidad: ejecución en Windows 10+ y Ubuntu 20.04+ sin modificaciones	Instalación y ejecución exitosa en las tres plataformas con el mismo código fuente
RNF-05	Usabilidad: tiempo de configuración inicial ≤ 10 minutos	Seguir la guía de instalación del README y completarla en el tiempo estipulado
RNF-06	Mantenibilidad: código estructurado en módulos independientes con docstrings en cada función	Cobertura de docstrings $\geq 80\%$ verificada con pydocstyle por ejemplo.

Table 4. – Requerimientos no funcionales del sistema.

2.2.2 Recursos a Utilizar

Hardware

Recurso	Especificación mínima	Justificación
Computador Laptop	Intel Core i5 (8ª gen+) o AMD Ryzen 5; 8 GB RAM; Windows 10 / Ubuntu 20.04	MediaPipe Pose opera en CPU sin GPU; la especificación mínima garantiza ≥ 20 FPS
Cámara web (webcam)	Resolución mínima 640×480 @ 30 FPS; interfaz USB 2.0 o integrada	Compatible con cv2.VideoCapture(0); cualquier cámara reconocida por el SO
Espacio en disco	500 MB libres para entorno virtual y dependencias	Python + OpenCV + MediaPipe + dependencias con tamaño aproximado de 450 MB
Conexión a internet	Necesaria solo para instalación inicial de paquetes pip	Requerida en Sprint 2 para descargar modelos BlazePose (mas o menos 25 MB)

Table 5. – Recursos de hardware requeridos.

Software y Librerías

2.2.3 Diseño Conceptual

El diseño conceptual del sistema se articula en torno a un pipeline de procesamiento en tiempo real compuesto por los cinco módulos definidos en el Sprint #1. El flujo completo se ejecuta en un bucle principal a la frecuencia de la cámara (30 FPS objetivo). El tiempo total de procesamiento por frame debe ser inferior a 50 ms para garantizar el requisito de ≥ 20 FPS. [?,?]

Librería / Herramienta	Versión	Licencia	Rol en el proyecto
Python	3.10+	PSF	Lenguaje principal de implementación
OpenCV (cv2)	4.8+	Apache 2.0	Captura, preprocesamiento, renderizado y visualización (M1, M4, M5) [?]
MediaPipe	0.10+	Apache 2.0	Detección de pose BlazePose; extracción de landmarks 11, 13, 15, 17, 19, 21 para ángulos articulares y apertura del gripper (M2, M3) [?, ?]
NumPy	1.24+	BSD	Álgebra vectorial para cálculo angular y manejo de arrays (M3)
Matplotlib	3.7+	PSF	Gráficas de evolución de ángulos articulares y métricas FPS (M5)
scikit-learn	1.3+	BSD	Métricas estadísticas: MAE, correlación de Pearson (evaluación opcional)
Git / GitHub	2.40+	GPL / MIT	Control de versiones y gestión del repositorio del proyecto

Table 6. – Stack de software y librerías del proyecto con versiones y licencias.

Arquitectura General del Pipeline

La figura ?? representa el flujo de datos del sistema desde la captura hasta la visualización.

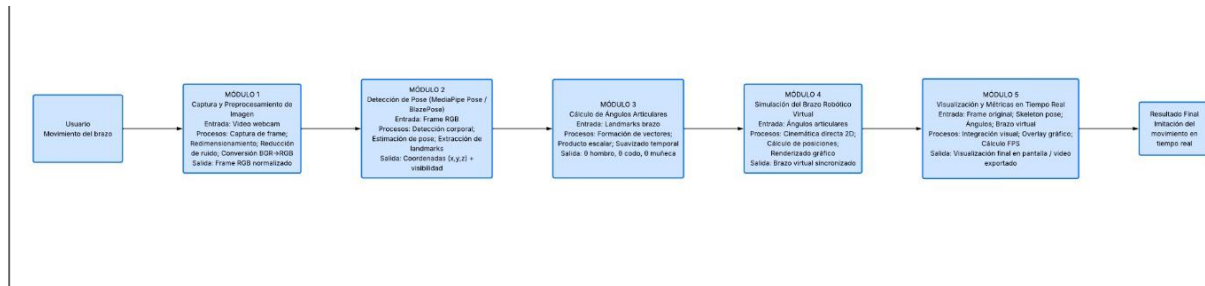


Figure 1. – Arquitectura conceptual del pipeline de procesamiento en tiempo real.

Diseño del Módulo M3 – Cálculo de Ángulos

El módulo M3 implementa el cálculo de ángulos mediante la siguiente lógica para cada articulación:

1. Extraer las coordenadas 2D (x, y) en píxeles de los landmarks proximal, central y distal de la articulación.
2. Construir los vectores $\mathbf{v}_1$ (de central a proximal) y $\mathbf{v}_2$ (de central a distal).
3. Calcular el ángulo mediante la ecuación ??:

$$\theta = \arccos\left(\frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{|\mathbf{v}_1| |\mathbf{v}_2|}\right) \quad (2.1)$$

El resultado en radianes se convierte a grados.

4. Aplicar filtro de media móvil de ventana $w = 5$ sobre el historial de ángulos de la articulación.

5. Retornar el ángulo suavizado junto con un flag de validez (`True` si visibilidad de todos los landmarks > 0.5).
6. Calcular la apertura del gripper: distancia euclidiana entre landmark 19 (punta índice) y landmark 21 (punta pulgar), normalizada entre un valor mínimo d_{min} (pinza cerrada) y máximo d_{max} (pinza abierta), expresada como porcentaje [0%, 100%].

Diseño del Módulo M4 – Cinemática Directa 2D

El brazo robótico virtual se modela como una cadena cinemática de tres eslabones más un efector final tipo gripper, anclada en una posición fija de la pantalla (base). Las longitudes de eslabón son parámetros configurables (en píxeles):

- $L_{hombro} = 120$ px (eslabón 1: base $\rightarrow$ articulación 1)
- $L_{antebrazo} = 100$ px (eslabón 2: articulación 1 $\rightarrow$ articulación 2)
- $L_{mano} = 60$ px (eslabón 3: articulación 2 $\rightarrow$ efector final)
- $L_{gripper} = 30$ px por rama (dos segmentos divergentes que representan la apertura de la pinza; el ángulo entre ellos es proporcional a $a_{gripper}$ calculado en M3)

Las posiciones cartesianas de cada junta se calculan mediante cinemática directa acumulativa (ecuaciones ??):

$$\begin{aligned}
 x_1 &= x_0 + L_{hombro} \cos(\theta_{hombro}) \\
 y_1 &= y_0 - L_{hombro} \sin(\theta_{hombro}) \\
 x_2 &= x_1 + L_{antebrazo} \cos(\theta_{hombro} + \theta_{codo}) \\
 y_2 &= y_1 - L_{antebrazo} \sin(\theta_{hombro} + \theta_{codo})
 \end{aligned} \tag{2.2}$$

2.2.4 Bocetos y Flujos de Pantalla

A continuación se describen los bocetos del sistema y los flujos de pantalla principales. Los bocetos representan la disposición visual de la interfaz de usuario en la ventana de visualización de OpenCV.

Boceto 1 – Pantalla Principal (Vista en tiempo real)

La ventana principal muestra el frame de video (640×480 px) con los siguientes elementos superpuestos:

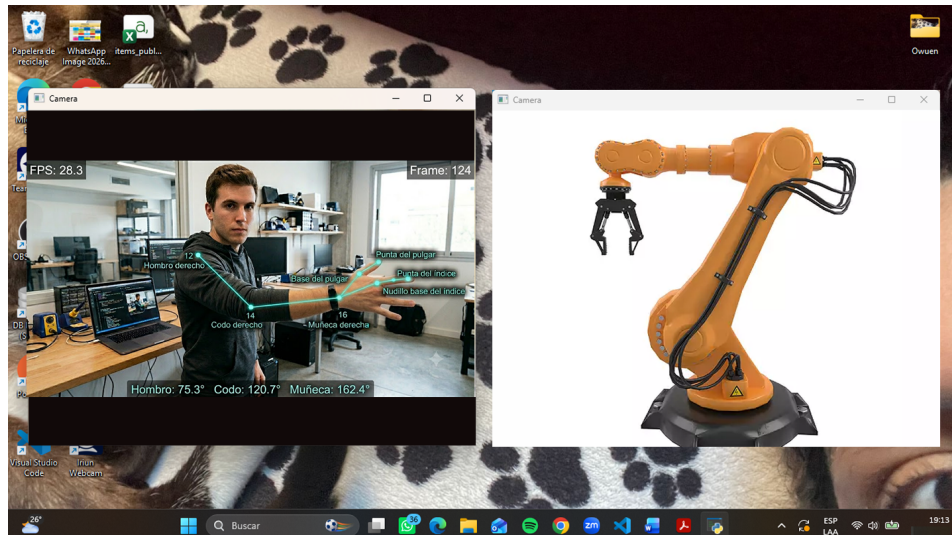


Figure 2. – Boceto de la pantalla principal del sistema (ventana OpenCV en tiempo real). Elementos: contador FPS, número de frame, skeleton MediaPipe con landmarks, brazo robótico virtual, ángulos articulares calculados en tiempo real.

Boceto 2 – Flujo de Pantalla del Sistema

El flujo de interacción del usuario con el sistema sigue la secuencia de estados de la figura ??.

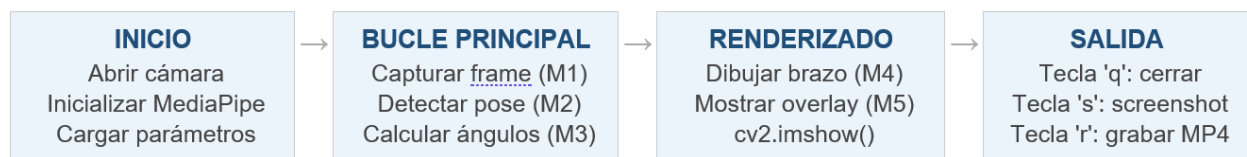


Figure 3. – Flujo de pantalla del sistema

2.3 Plan de Implementación del Proyecto

El plan de implementación sigue la metodología SCRUM con tres sprints. El Sprint #1 (semanas 1–2) cubrió la definición del problema. El Sprint #2 (semanas 3–4) cubre la metodología. El Sprint #3 (fin del curso DIP) abarca el desarrollo completo, análisis de resultados y conclusiones. [?]

Sprint	Semanas	Actividades principales	Entregable
Sprint #1	1–2	Revisión del estado del arte (tema macro, componentes y herramientas). Definición del diagrama de bloques y módulos. Definición de objetivos, entradas y salidas por módulo.	Informe #1 (Secciones 1.1–1.5)
Sprint #2	3–4	Análisis de datos (datasets, características, preprocesamiento). Análisis de confiabilidad y compatibilidad. Definición de requerimientos, recursos, diseño conceptual y bocetos. Plan de implementación.	Informe #2 (Sección 2)
Sprint #3 – Fase 1	5–7	M1: captura y preprocesamiento (cv2.VideoCapture, normalización, filtro gaussiano). M2: integración MediaPipe Pose y extracción de landmarks 11–16. M3: cálculo vectorial de ángulos con filtro media móvil y pruebas unitarias. Incluye cálculo de apertura del gripper por distancia euclidiana landmarks 19–21.	Código M1–M3 funcional + pruebas unitarias
Sprint #3 – Fase 2	8–10	M4: cinemática directa 2D, renderizado del brazo y simulación visual del gripper (apertura/cierre). M5: visualización integrada y FPS overlay. Integración de M1–M5 en pipeline completo.	Sistema integrado funcional a ≥ 20 FPS
Sprint #3 – Fase 3	11–12	Evaluación del sistema: FPS, MAE angular, robustez ante iluminación. Análisis estadístico con scikit-learn y gráficas con matplotlib. Redacción de conclusiones, recomendaciones y futuros trabajos.	Informes #3 y #4 (Secciones 3 y 4)

Table 7. – Plan de implementación del proyecto por sprint y semanas.

2.3.1 Gestión de Riesgos

Se identifican los siguientes riesgos técnicos con sus planes de mitigación:

Riesgo	Probabilidad	Impacto	Mitigación
FPS < 20 en hardware de bajo rendimiento	Media	Alto	Reducir resolución a 320×240 ; desactivar skeleton completo; usar MoveNet como fallback
Oclusión del brazo en poses extremas	Alta	Medio	Implementar umbral de visibilidad (> 0.5); mantener último ángulo válido cuando visibilidad es baja
Ruido excesivo en landmarks por iluminación deficiente	Media	Medio	Aumentar ventana de media móvil a $w = 10$; Inversión de capital en mejorar condiciones de iluminación
Incompatibilidad de versiones de librerías	Baja	Alto	Fijar versiones exactas en requirements.txt; usar entorno virtual Python aislado

Table 8. – Identificación de riesgos técnicos y planes de mitigación.

References

- [1] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, “Blazepose: On-device real-time body pose tracking,” *arXiv*, 2020.
- [2] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft coco: Common objects in context,” in *Computer Vision – ECCV 2014* (D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, eds.), (Cham), pp. 740–755, Springer International Publishing, 2014.
- [3] A. S. B. Pauzi, F. B. M. Nazri, S. Sani, A. M. Bataineh, M. N. Hisyam, M. H. Jaafar, M. N. Ab Wahab, and A. S. A. Mohamed, “Movement estimation using mediapipe blazepose,” in *Advances in Visual Informatics. IVIC 2021*, vol. 13051 of *Lecture Notes in Computer Science*, pp. 590–600, Springer, Cham, 2021.
- [4] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, “Mediapipe: A framework for building perception pipelines,” *arXiv*, 2019.
- [5] G. Bradski, “The opencv library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [6] B. X. Vintimilla, “Propuestas de proyectos – curso dip 2026 pao-i.” Diapositivas de presentación, 2026. Escuela Superior Politécnica del Litoral (ESPOL), Facultad de Ingeniería en Electricidad y Computación (FIEC).